OSCAL Builder Session

Prerequisites Guide

GRC Engineering Club

Complete these steps before the session. Budget 30-45 minutes.

1. Clone the Repository

```
git clone https://github.com/GRCJP/oscal-pipeline-workshop.git
cd oscal-pipeline-workshop
```

2. Python 3

You need Python 3.10 or newer.

Check if installed:

```
python3 --version
```

Install if needed:

- macOS:

```
brew install python3
```

- Windows: Download from https://www.python.org/downloads/
- Linux:

```
sudo apt install python3 python3-pip
```

Install required packages:

```
pip3 install openpyxl python-docx python-dotenv boto3 pillow requests
```

3. AWS Free Tier Account

If you don't have one: https://aws.amazon.com/free/

Create an IAM user for the workshop

1. Sign in to the AWS Console as root or admin
2. Go to IAM > Users > Create user

3. Name: workshop-pipeline
4. Attach policy: ReadOnlyAccess
5. Create an access key (select "Command Line Interface" use case)
6. Save the Access Key ID and Secret Access Key - you'll need them for .env

Enable AWS Config

> **IMPORTANT: Make sure you are in us-east-1 (check the region selector in the top right of the console).**

1. Go to AWS Config in the console
2. Click Get started or Settings
3. Select Record all resource types
4. For S3 bucket, create a new one or use an existing bucket
5. Let AWS create the service-linked role automatically
6. Click Confirm

This gives the pipeline a complete inventory of your AWS resources.

Alternative: Run the setup script

```
bash scripts/aws-setup.sh
```

This creates all AWS resources in us-east-1 automatically.

Verify AWS CLI works

> **All workshop resources must be in us-east-1 (N. Virginia). Using a different region will cause the pipeline to miss your resources.**

```
brew install awscli   # macOS
# or: pip3 install awscli

aws configure
# Enter your Access Key ID, Secret Access Key, and region: us-east-1

# Test connection:
aws iam list-users
```

4. GitHub Personal Access Token

1. Go to https://github.com/settings/tokens
2. Click Generate new token (classic)
3. Select scopes: repo, read:org
4. Copy the token - you'll need it for .env

Test it:

```
export GITHUB_TOKEN=ghp_your_token_here
gh auth status
```

5. Prowler (Open Source Cloud Security Scanner)

```
pip3 install prowler
```

Verify:

```
prowler --version
```

Prowler uses your AWS credentials from .env. No separate account needed.

6. Trivy (Container & IaC Scanner)

macOS:

```
brew install trivy
```

Linux:

```
sudo apt-get install wget apt-transport-https gnupg lsb-release
wget -qO - https://aquasecurity.github.io/trivy-repo/deb/public.key | sudo apt-key add -
echo "deb https://aquasecurity.github.io/trivy-repo/deb $(lsb_release -sc) main" | \
  sudo tee /etc/apt/sources.list.d/trivy.list
sudo apt-get update && sudo apt-get install trivy
```

Windows:

```
choco install trivy
```

Verify:

```
trivy --version
```

7. Linear API Key (POA&M Issue Tracking)

Linear is used to create trackable issues from pipeline findings, each with evidence screenshots attached.

1. Go to https://linear.app/settings/api
2. Click Create key
3. Copy the API key - you'll need it for .env

Find your team key:

Your team key is the short prefix on issues (e.g., "GRC" if issues are GRC-1, GRC-2). Check it under Settings > Teams.

8. Configure Your Environment

1. Copy the example environment file:

```
cp prereqs/.env.example .env
```

2. Open .env and fill in your credentials:

```
open .env        # macOS
notepad .env     # Windows
nano .env        # Linux/terminal
```

3. Load the environment:

```
export $(cat .env | grep -v '^#' | xargs)
```

9. Verify Everything Works

Run the converter to confirm your setup:

```
python3 scripts/excel_to_oscal.py \
  --input Templates/fedramp-moderate-template-ssp.xlsx \
  --output oscal
```

You should see 57 controls processed and CONVERSION COMPLETE.

Run the full pipeline (all 6 stages end-to-end):

```
python3 scripts/run_pipeline.py --github-repo oscal-pipeline-workshop
```

This runs: Convert, Discover, Assess, Reconcile, Enforce, and Report (including Linear export with evidence screenshots).

Tool Summary

Tool	Purpose	Cost	Install
Python 3	Run pipeline scripts	Free	brew install python3
boto3	AWS SDK for Python	Free	pip3 install boto3
python-dotenv	Load .env credentials	Free	pip3 install python-dotenv
AWS CLI	Query AWS APIs	Free tier	brew install awscli
AWS Config	Asset discovery & inventory	Free tier	AWS Console
GitHub + Token	Source control evidence	Free	github.com/settings/tokens

Linear	POA&M issue tracking	Free	linear.app/settings/api
Prowler	Cloud security posture	Free (OSS)	pip3 install prowler
Trivy	Container/IaC scanning	Free (OSS)	brew install trivy
NVD API	Vulnerability intelligence	Free	No install needed
CodeQL	SAST code scanning	Free	Runs via GitHub Actions

Troubleshooting

`python: command not found`

Use python3 instead of python. macOS does not ship with python.

`aws: command not found`

Install AWS CLI: brew install awscli or pip3 install awscli

`AWS access denied errors`

Check that your IAM user has ReadOnlyAccess policy attached.

`GitHub token not working`

Make sure you selected repo and read:org scopes when creating the token.

`Linear issues not creating`

Check that LINEAR_API_KEY and LINEAR_TEAM_KEY are set in .env. The team key is the short prefix on your issues (e.g., GRC).

Questions?

Reach out in the GRC Engineering Club channel before the session.